Лабораторная работа №8.
Применение объектов JavaScript при обработке Web-форм

Цель работы: изучить синтаксис объектов String, Math, Date, Array, а также пользовательских объектов, способов задания их свойств и методов, применить свойства и методы объектов для решения прикладных задач.

Теоретический материал

Объекты ядра языка JavaScript расширяют возможности Web – программиста, у которого появляется возможность высокоуровневой обработки строк, выполнения математических функций, работы с хронологическими датами и массивами. Все это делается с помощью встроенных объектов String, Math, Date, Array.

Важной особенностью указанных объектов является то, что они могут использоваться как в серверных сценариях, так и сценариях выполняемых браузером (клиентские сценарии). Можно утверждать, что данные объекты незаменимы в Web – программировании.

Строковые объекты JavaScript

Строковые литералы представляют собой произвольную последовательность символов в двойных или одинарных кавычках. Однако, в JavaScript это еще объекты типа String, имеющие свойства и многочисленные методы.

Строковый объект создается подобно всем другим объектам:

var stringName = new String(Строка);

Экземпляр строкового объекта является переменной, которая ссылается на строку и для которой оператором new с помощью конструктора String (конструктор всегда имеет то же имя, что и объект, но может вызываться как функция с параметрами) выделяется необходимая память. В отличие от всех других объектов, для строковых объектов возможна более привычная форма создания:

var stringName = Строка;

Например, обе следующие конструкции в JavaScript работают одинаково:

var str1= new String(“Строковый литерал”);

var str2= “Строковый литерал”;

Однако конструкция с конструктором работает в серверных сценариях стабильнее. Важнейшим свойством объекта String является length, которое возвращает количество символов в строке.

Следующий пример демонстрирует использование этого свойства. При загрузке страницы появляется стандартное окно ввода, в котором вводится некоторая строка (рисунок 1). После ввода строки закрытия окна, выводится вся страница, в которой напечатана введенная строка и ее длина (рисунок 2).

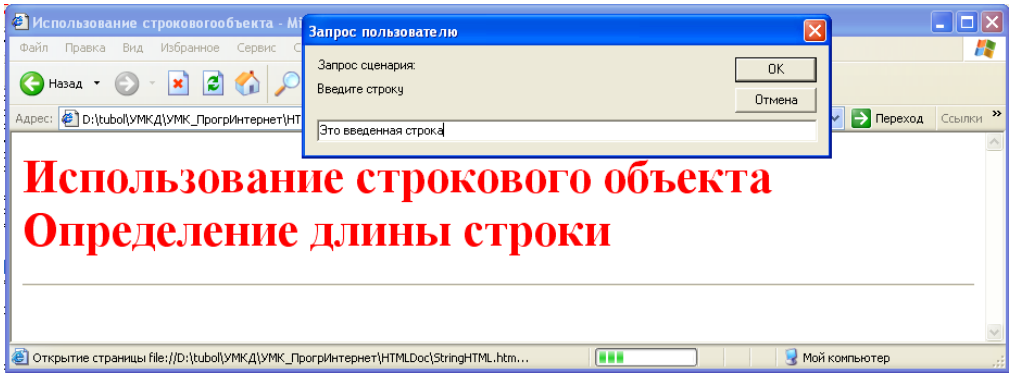


Рисунок 1. При загрузке страницы в стандартном окне вводится некоторая строка

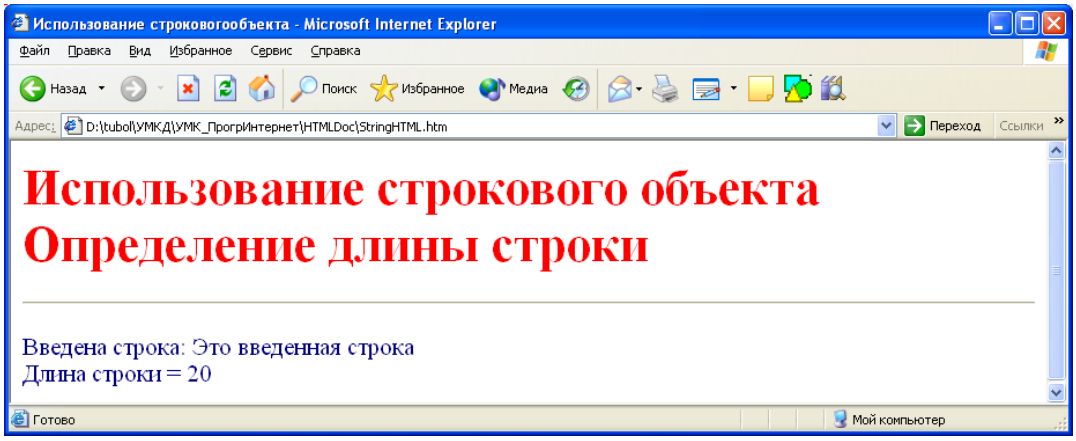


Рисунок 2. Сценарий анализирует строковый объект и выводит его длину (количество введенных символов, включая пробелы)

Для реализации данного примера используются три файла.
Файл StringHTML.htm – содержит следующий HTML – код:

```
<html>
  <head>
    <title>Использование строкового объекта</title>
    <script type="text/javascript" src="StringJS.js"></script>
    <style type="text/css">@import: url("StringCSS.css");</style>
  </head>
  <body>
    <h1>Использование строкового объекта<br>Определение
    длины строки</h1>
    <hr size="2">
    <script>SetStr();</script>
  </body>
</html>
```

Файл StringJS.js - содержит код сценария:

```
function SetStr(){
  var str= prompt("Введите строку");
  document.writeln("<p>Введена строка: "+str+"<br>");
  document.writeln("Длина строки = "+str.length+"</p>");
}
```

Файл StringCSS.css – таблицу стилей:

h1 {color: red}

p {color: navy}

Сценарий содержит определение функции SetStr, которая вызывается в девятой строке HTML – страницы: <script>SetStr();</script> Маленький сценарий в 9-й строке нужен исключительно для того, чтобы вызвать функцию SetStr, поскольку внутри HTML–кода операторы JavaScript применять нельзя.

Объект String имеет большое число методов – функций, определенных внутри объекта.

Таблица 1. Основные методы объекта String

Метод	Пример	Результат
anchor()	String.anchor(Name)	 String
big()	String.big()	<big> String </big>
bold()	String.bold()	String
charAt()	String.charAt(Index)	Возвращает символ строки в позиции Index
fontcolor()	String.fontcolor(Color)	 String
fontsize()	String.fontsize(Size)	 String
indexOf()	String.indexOf(Substring, Start)	Поиск в строке String подстроки Substring, начиная с позиции Start
italics()	String.italics()	String
lastindexOf()	String.lastindexOf(Substring, Start)	Поиск последнего вхождения в строке String подстроки Substring, начиная с позиции Start
link()	String.link(URL)	 String
replace()	String.replace(findStr, replaceStr)	Замена в строке String подстроки findStr на подстроку replaceStr
small()	String. small()	String
split()	String.split(Separator)	Разбивает строку по символу Separator, возвращая строковый массив
strike()	String.strike()	<strike> String </strike>
sub()	String.sub()	_{String}
substring()	String.substring(Start, End)	Возвращает подстроку,

		начиная с позиции Start и заканчивая End
sup()	String.sup()	^{String}
toLowerCase()	String.toLowerCase()	Преобразует все символы строки String в символы нижнего регистра
toUpperCase()	String.toUpperCase()	Преобразует все символы строки String в символы верхнего регистра

Поиск в строке String подстроки Substring начинается с позиции Start (если не задан – поиск с начала). Методы поиска возвращают -1, если поиск не дал результата, иначе – позицию начала подстроки.

Рассмотрим пример использования методов объекта String. При загрузке страницы запрашивается позиция начала подстроки во введенной пользователем строке, а затем она выделяется.

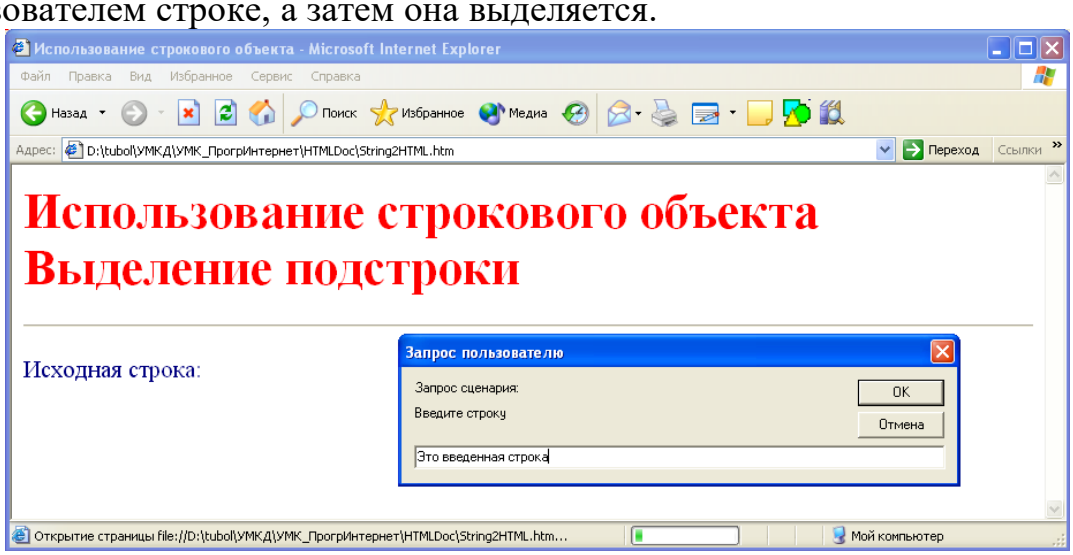


Рисунок 3. Окно ввода некоторой исходной строки

Для реализации данного примера необходимо создать следующие файлы:

1. файл String2HTML.htm – HTML-документ;
2. файл String2JS.js – сценарий;
3. файл String2CSS.css – таблица стилей;

Файл String2HTML.htm содержит следующий код:

```
<html>
  <head>
    <title>Использование строкового объекта</title>
    <script type="text/javascript" src="String2JS.js"></script>
    <style type="text/css">@import: url("String2CSS.css");</style>
  </head>
  <body>
    <h1>Использование строкового объекта<br>Выделение
    подстроки</h1>
```

```
    <hr size="2">
    <p>Исходная строка: <script>SetStr();</script></p>
    <p><script>GetSubStr();</script></p>
  </body>
</html>
```

В данном коде помещено 2 вызова пользовательских функций JavaScript сначала для ввода строки, а затем для ввода позиции выделения.

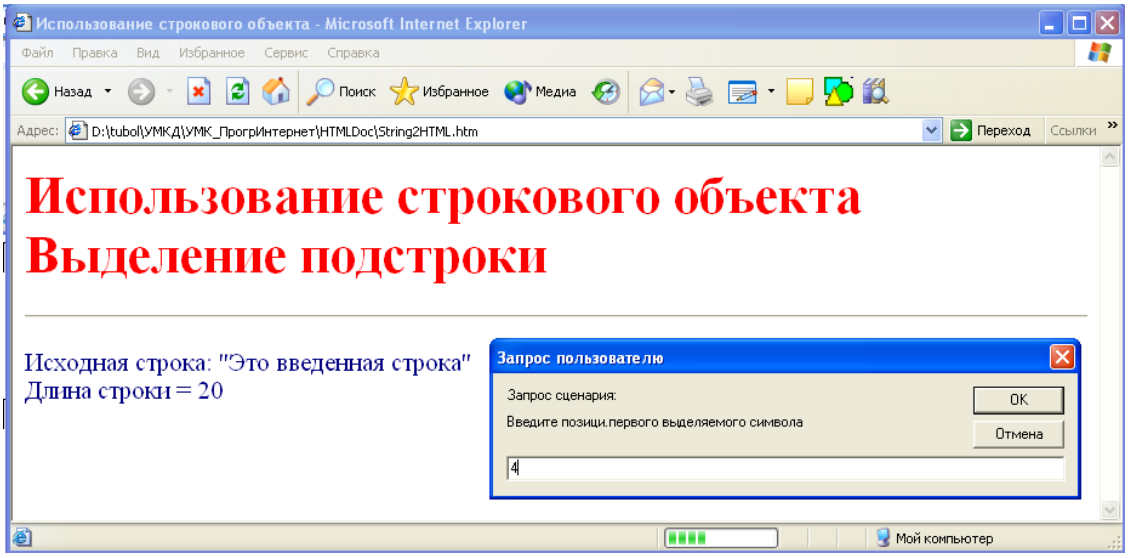


Рисунок 4. Окно запроса позиции начала выделения подстроки в исходной строке

В файле String2JS.js содержится сценарий, который содержит определения двух функций SetStr и GetSubStr. Первая вводит строку, вторая – позицию выделения:

```
var str;
function SetStr(){
  str= prompt("Введите строку");
  document.writeln("\""+str+"\"<br>");
  document.writeln("Длина строки = "+str.length+"</p>");
}
function GetSubStr(){
  var ns= prompt("Введите позицию первого выделяемого
  символа");
  var n=ns*1;
  document.writeln("Выделение начинается после позиции: "
  +ns+"<br>");
  var substr=str.substring(n);
  document.writeln("Выделена подстрока: \""+substr+"\"<br>");
  document.writeln("Длина подстроки = "+substr.length+"</p>");
}
```

Кроме этого, обе функции с помощью метода writeln объекта document выводят на страницу некоторую дополнительную информацию о длине исходной строки, позиции выделения и длине выделенной подстроки. На рисунке 5 показана страница после полной загрузки браузером.

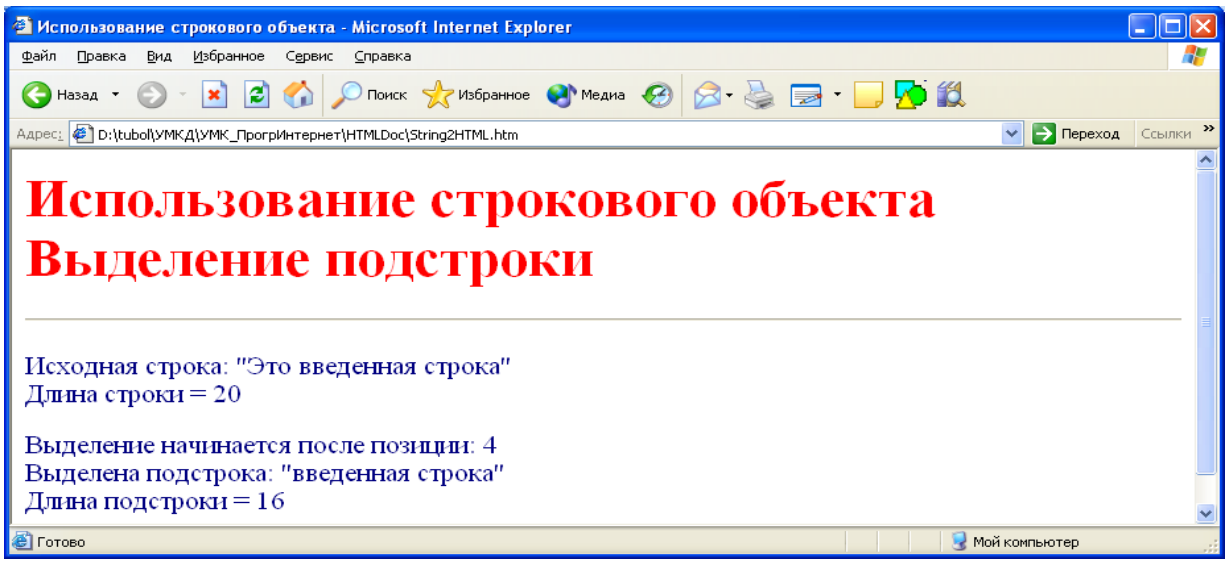


Рисунок 5. Окно со страницей результатов

Отметим, что после загрузки страницы браузером не могут выполняться никакие скрипты, кроме обработчиков событий.

Использование строковых объектов значительно облегчает работу со строками, и они используются в сценариях постоянно.

Массивы JavaScript

Массивы JavaScript представляют собой упорядоченную совокупность однотипных литералов. Фактически каждый элемент массива является переменной. По сравнению с универсальными языками программирования (например, C++) массивы JavaScript обладают большими возможностями, поскольку являются объектами, точнее экземплярами объекта Array.

Объявление массива выглядит следующим образом:

var ar= new Array(Values);

Создается переменная являющаяся ссылкой на массив, значения которого Values можно задать внутри конструктора. Например, следующий код задает числовой массив с пятью элементами:

var ar= new Array(1,2,3,4,5);

После того, как массив объявлен, к его элементам можно обращаться по номеру (нумерация элементов начинается с нуля). Массив можно также задавать не с помощью значений, а указанием количества элементов. Так объявляется массив из 10 элементов (тип которых пока не определен):

var ar= new Array(10);

В данном случае сценарию становится известно, что переменная ar представляет собой массив из 10 элементов, значения которых пока не заданы, но могут быть заданы позднее. Для работы с массивами (и, в частности, инициализации) обычно используются циклы.

Массивы JavaScript подобно другим объектам имеют свойства и методы. Главным свойством массива является length, возвращающее количество элементов в массиве. Это свойство является не тривиальным, поскольку массивы JavaScript являются динамическими: в них можно на этапе выполнения добавлять и удалять элементы. Основными методами являются:

Таблица 2. Основные методы объекта Array

Метод	Синтаксис	Описание
concat()	Array1.concat(Array2)	В конец массива Array1 добавляет элементы массива Array2
join()	Array.join(Separator)	Объединяет элементы массива в строку, используя разделитель символ Separator (например, «/»)
pop()	Array.pop()	Удаляет последний элемент массива Array
push()	Array.push(Value)	Добавляет в конец массива Array новый элемент со значением Value
shift()	Array.shift()	Удаляет первый элемент и возвращает его значение
slice()	Array.slice(Start, End)	Выделяет и возвращает фрагмент массива Array от позиции Start до позиции End
sort()	Array.sort()	Сортирует массив Array
splice()	Array.splice(Start, N, Values)	Удаляет N элементов, начиная с позиции Start, и заменяет их значениями Values
unshift()	Array.unshift(Value)	Добавляет элемент в начало массива Array со значением Value

Рассмотрим следующий пример использования массива. При загрузке страницы инициализируется (в цикле) массив и его элементы выводятся на страницу:

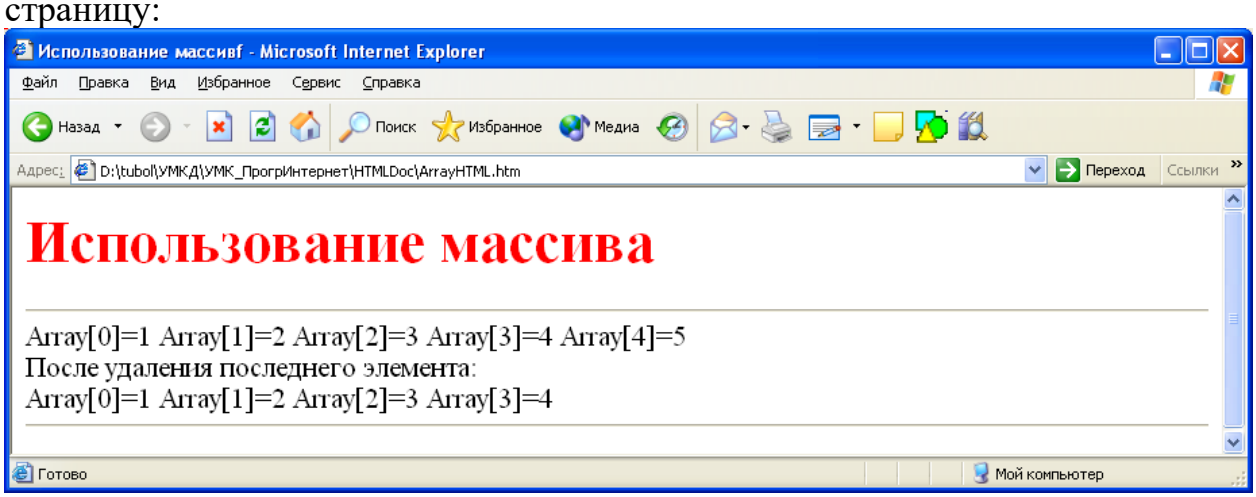


Рисунок 6. Окно со страницей, содержащей элементы массива

Кроме обычного файла с каскадной таблицей стилей, используется файл HTML–документа ArrayHTML.htm:

```
<html>
  <head>
     <title>Использование массива</title>
     <style type="text/css">@import:url("ArrayCSS.css");</style>
  </head>
```

```
<body>
    <h1>Использование массива</h1>
    <hr size="2">
    <script type="text/javascript">
        var ar= new Array(1,2,3,4,5);
        var i;
        for (i=0; i< ar.length; i++){
           document.write("  Array["+i+"]="+ar[i]);
        }
        document.write("<br>");
        document.write("После удаления последнего
        элемента:<br>");
        ar.pop();
        for (i=0; i< ar.length; i++){
           document.write("  Array["+i+"]="+ar[i]);
        }
    </script>
    <hr size="2">
</body>
</html>
```

Сценарий в строках 8-18 выполняет необходимые действия с массивом. Сначала (в строке 9) задается массив. Затем (строки 11-12) массив выводится на страницу; в строке 15 удаляется последний элемент; а затем (в строках 16-17) массив снова выводится на страницу. Остальные строки в комментариях не нуждаются, осуществляя очевидные действия.

Отметим, что массивы в JavaScript применяются постоянно, поскольку они являются динамическими объектами, крайне полезными при создании сценариев.

Математический объект и объекты хронологических дат JavaScript

Хотя язык JavaScript создавался не для написания сложных вычислительных программ, но и в нем без математических вычислений не обойтись. Минимально достаточный набор математических функций и констант скомпонован в виде объекта Math, который всегда используется напрямую (экземпляры объектов для него не создаются).

Объект Math имеет следующие свойства, которые инкапсулируют наиболее часто используемые математические константы:

Таблица 3. Свойства (математические константы) объекта Math

Свойство	Синтаксис	Значение
E	Math.E	Второй замечательный предел e
LN2	Math.LN2	Натуральный логарифм ln2
LN10	Math.LN10	Натуральный логарифм ln10
LOG2E	Math.LOG2E	Двоичный логарифм e
LOG10E	Math.LOG10E	Десятичный логарифм e
PI	Math.PI	Число «пи»

SQRT1_2	Math.SQRT1_2	Корень квадратный из 1/2
SQRT2	Math.SQRT2	Корень квадратный из 2

Кроме математических констант, методы объекта Math представляют (инкапсулируют) наиболее часто используемые математические функции:

Таблица 4. Методы (математические функции) объекта Math

Метод	Синтаксис	Значение
abs()	Math.abs(x)	Абсолютная величина числа x
acos()	Math.acos(x)	arccoc(x)
asin()	Math.asin(x)	arcsin(x)
atan()	Math.atan(x)	arctan(x)
ceil()	Math.ceil(x)	Округление до меньшего целого
cos()	Math.cos(x)	cos(x)
exp()	Math.exp(x)	e^x
floor()	Math.floor(x)	Округление до большего целого
log()	Math.log(x)	Натуральный логарифм
max()	Math.max(x,y)	Максимум двух чисел
min()	Math.min(x,y)	Минимум двух чисел
pow()	Math.pow(x,y)	x^y
random()	Math.random(x)	Случайное число от 0 до 1
round()	Math.round(x)	Округление до ближайшего целого
sin()	Math.sin(x)	sin(x)
sqrt()	Math.sqrt(x)	Корень квадратный их x
tan()	Math.tan(x)	Тангенс x

Кроме математических действий, часто требуется осуществлять различные операции с хронологическими датами; без этого не может обойтись практически ни одно приложение для бизнеса.

Хронологические даты представляют собой экземпляры объекта Date, и имеют целочисленное внутреннее представление в виде числа миллисекунд, прошедших с 1.1.1970. Ссылка на объект хронологической дата (переменная типа Date) создается следующим образом:

var dt = new Date(год, месяц, день);

Существует еще несколько конструкторов хронологических дат, но этот используется чаще всего. Конструктор по умолчанию (с пустыми скобками) создает переменную типа Date, которая ссылается на текущую (на момент создания) дату. Как и любые объекты, объекты хронологических дат обладают свойствами и методами. Наиболее часто используются методы объект Date:

Таблица 5. Основные методы объекта Date

Метод	Значение
getDate()	Возвращает число месяца (1 - 31)
getDay()	Возвращает день недели от 0 (воскресенье) до 6 (суббота)
getMonth()	Возвращает номер месяца 0 (январь)

getFullYear()	Возвращает номер года всегда четырехзначный
getYear()	Возвращает номер года
setDate()	Устанавливает число месяца (1 - 31)
setMonth()	Устанавливает номер месяца 0 (январь)
setYear()	Устанавливает номер года

Указанных методов обычно достаточно для создания практически любых сценариев. Сценарий, работающий на следующей странице, при загрузке запрашивает некоторую дату, и выводит наименование дня недели, который приходится на эту дату.

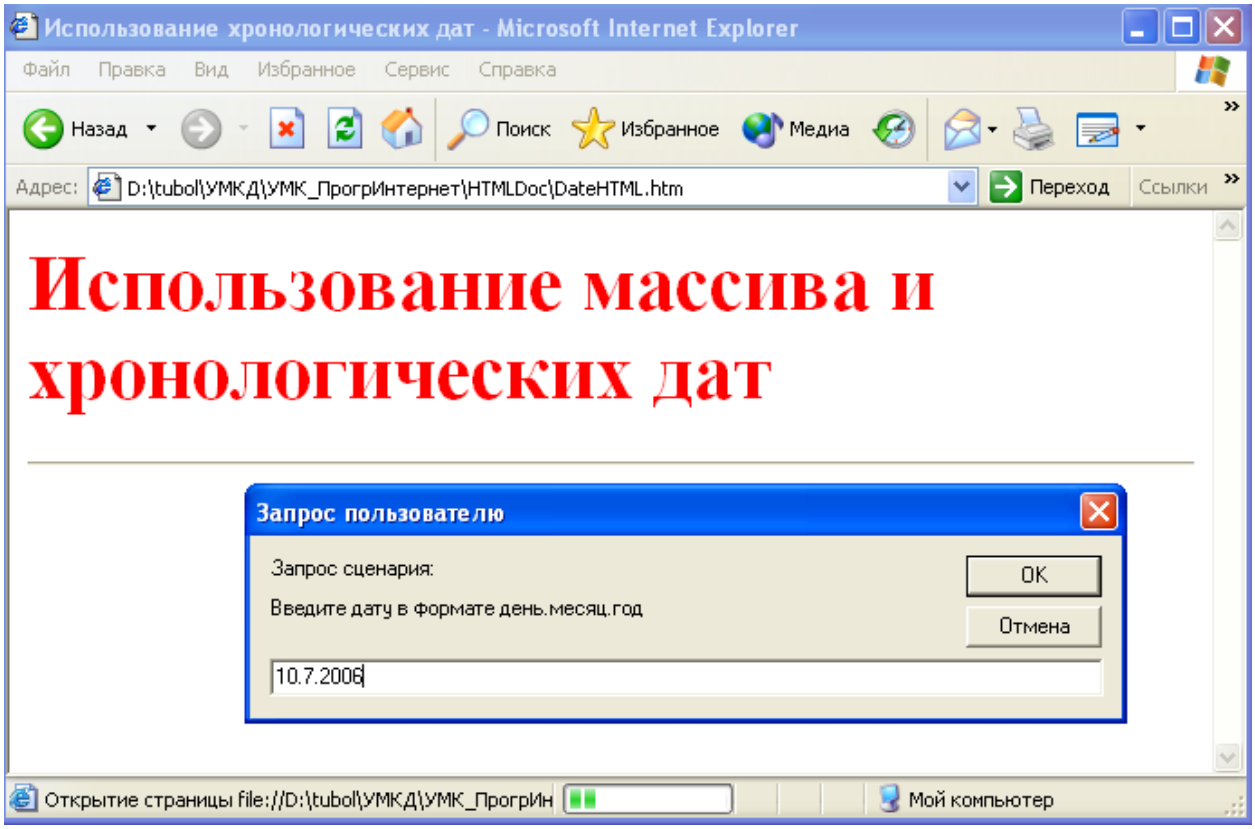


Рисунок 7. Окно ввода даты при загрузке страницы

Поскольку сценарий достаточно прост, он весь реализован в файле HTML-страницы (DateHTML.htm):

```
<html>
  <head>
    <title>Использование хронологических дат</title>
    <style type="text/css">@import: url("DateCSS.css");</style>
  </head>
 <body>
  <h1>Использование массива и хронологических дат</h1>
  <hr size="2">
  <script type="text/javascript">
      var ar_sd= new
      Array("Воскресенье","Понедельник","Вторник","Среда","Ч
      етверг","Пятница","Суббота");
      var sd= prompt("Введите дату в формате день.месяц.год");
```

```
        var ar_dat = sd.split(".");
        var dat = new Date(ar_dat[2]*1,ar_dat[1]*1-1,ar_dat[0]*1);
        document.write("<p>Введена дата: "+sd+"<br>");
        document.write("Это: "+ ar_sd[dat.getDay()]+" </p>");
    </script>
    <hr size="2">
  </body>
</html>
```

Разберем работу сценария (строки 8-15). В строке 9 задается символьный массив с названиями дней недели на русском языке. В 10-й строке запрашивается дата, она будет получена в виде строкового литерала. В 11-й строке дата разлагается на составляющие, чтобы затем в 12-й строке скрипта создать объект хронологических дат (число месяца пришлось уменьшить на 1, т.к. нумерация месяцев ведется с 0). В 14-й строке определен и выведен день недели.

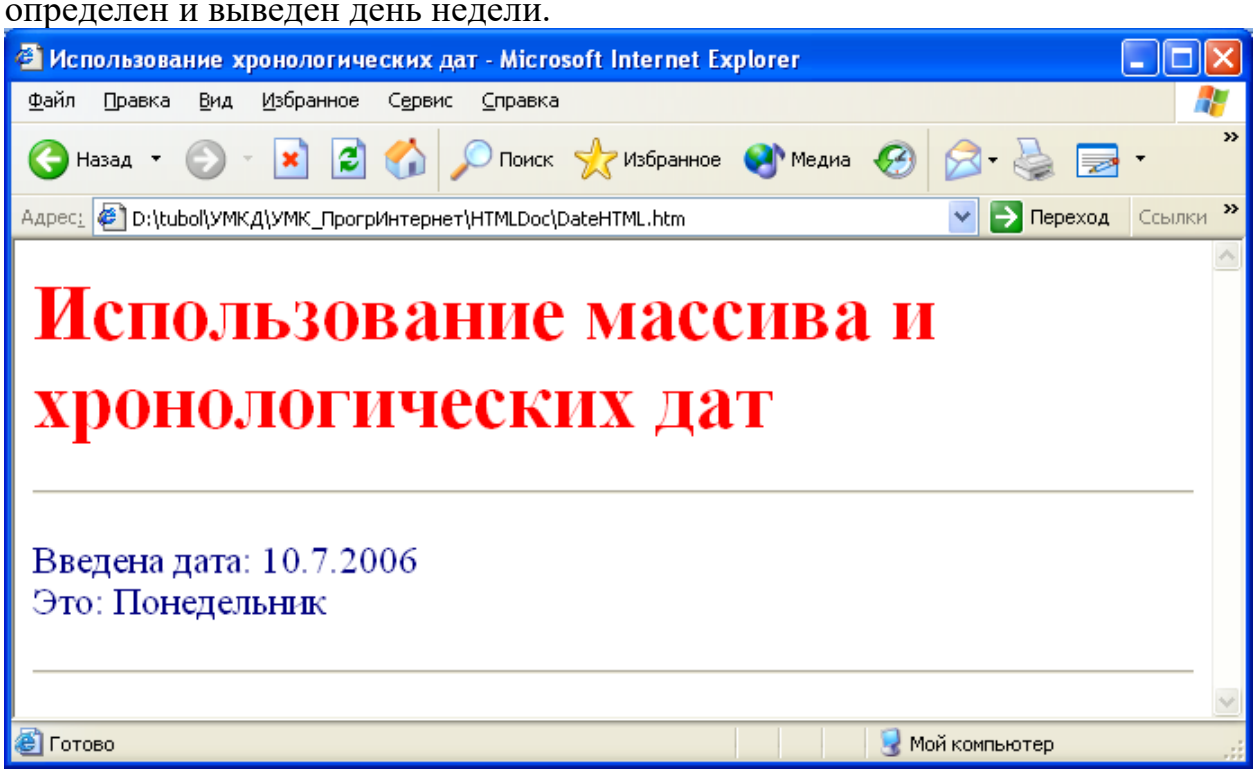


Рисунок 8. Окно с информацией о дне недели, приходящемся на заданную дату

Отметим, что работа с хронологическими датами характерна для бизнес-приложений в Интернет.

Рассмотренные объекты String, Math, Date, Array являются встроенными объектами JavaScript. Однако в JS можно создавать свои объекты. Объект представляет собой удобное средство организации данных и функций. Это контейнер, содержащий переменные, называемые свойствами, и функции, называемые методами. Применение объекта в программах состоит в манипулировании его свойствами и методами.

Объекты в JavaScript можно создать несколькими способами. Мы рассмотрим три таких способа:

1. **function имя_конструктора([пар1,...[, парN])**

{ код}

имя_объекта=new имя_конструктора (["пар1",...[, "парN"])

2. **имя_объекта =new Object()**

имя_объекта.свойство=значение

3. **имя_объекта={свойство1:значение1, свойство2:значение2,…}**

Для обращения к свойствам и методам объекта используется следующий синтаксис:

ссылка_на_объект.свойство

ссылка_на_объект.метод{[параметры]}

Рассмотрим указанные выше способы создания объектов более подробно. Первый способ основан на функции, в теле которой описываются все свойства и методы создаваемого объекта. Поскольку эта функция играет определяющую роль в создании объекта, ее называют функцией-конструктором или просто конструктором объекта. Что, собственно, должен делать конструктор? Очевидно, он должен ввести имя создаваемого объекта, а также его свойства и методы. Кроме того, он должен допускать возможность присваивания начальных значений свойствам. Имя функции-конструктора объекта является одновременно и именем создаваемого объекта. Свойства и методы создаваемого объекта задаются в теле функции-конструктора с помощью операторов присваивания. При этом имена переменных-свойств записываются с ключевым словом this(этот):

this.переменная

Рассмотрим в качестве примера функцию-конструктор, определяющую объект car (автомобиль) со свойствами name (название), model (модель) и color (цвет):

```
function car(name, model, color){
  this.name=name
  this.model=model
  this.color=color
}
```

Эта функция определяет объект car. Чтобы создать конкретный экземпляр объекта car, следует выполнить выражение с вызовом этой функции, которой можно передать значения параметров. Мы уже знаем, что экземпляр объекта создается с помощью оператора присваивания с ключевым словом new:

```
Mycar=new car("Лада", "ВА321099", "green")
```

Итак, мы создали объект Mycar, являющийся экземпляром объекта car. Таких экземпляров с различными именами можно создать несколько. Значения свойств объекта Mycar можно изменять в программе:

```
Mycar.model  //значение равно "ВА321099"
Mycar.name  // значение равно "Лада"
Mycar.model="ВА32110"  // значение равно "ВА32110"
```

Как видите, объект – это просто особый способ группировки данных и их использования (составное имя переменной: объект.свойство).

Второй способ основан на использовании конструктора new Object()

```
Mycar= new Object()
Mycar.name ="Лада"
Mycar.model ="BA321099"
Mycar.color="green"
```

В этом способе создаваемый объект является экземпляром встроенного объекта Object() подобно тому, как, например, создаваемый массив является экземпляром объекта Аггау. Объект Object() является корневым для всех ранее рассмотренных встроенных объектов.

В третьем способе применяется следующая компактная запись определения объекта:

```
Mycar={name:"Лада", model: "BA321099", color:" green "}
```

Здесь все определение свойств заключается в фигурные скобки. Пары свойство-значение разделяются запятыми, а имя свойства отделяется от значения двоеточием.

При создании объекта мы можем задать значения свойств по умолчанию, т. е. значения, которые будут иметь свойства, если при создании экземпляра этого объекта значения его свойств не указаны явным образом (т. е. имеют значения null, 0 или ""). Это делается с помощью логического оператора ИЛИ (обозначаемого ||) в конструкторе объекта в виде функции:

```
function car(name,model,color){
  this.name=name || " неизвестно "
  this.model=model || "неизвестно"
  this.color =color || "black"
}
```

Существует два способа для описания собственных методов:

1. Описание метода включается в описание класса;

2. С использованием свойства prototype, которое позволяет добавлять новые методы к существующему классу.

имяКласса.prototype.имяМетода = function(список параметров метода)

Общая постановка задачи

1. Выполнить два индивидуальных задания по варианту.

2. Поместить разработанный **JS**-код сценария на FTP-сервере.

3. Составить электронный отчет о проделанной работе. На титульном листе отчета указать URL для запуска разработанной программы на выполнение.

Список индивидуальных данных

Задание №1

1. Создать страницу, содержащую приветствие в виде: «Сегодня - 10 сентября 2019 года, 13:01:25, вторник». Для этого используйте объекты Date и Array. Кроме того, страница, должна содержать форму, для вычисления площади треугольника по 2-м сторонам и углу между ними.
2. Создать страницу, содержащую приветствие в виде: «Сегодня - 10 сентября 2019 года, 13:01:25, вторник». Для этого используйте объекты Date и Array. Кроме того, страница, должна позволять ввести элементы массива и вывести минимальный из них. Форма должна содержать с 2-мя текстовыми полями и 2-мя кнопками типа «submit». 1-е текстовое поле - для ввода элементов массива , кнопка «Ввод элемента» для подтверждения ввода элемента массива (для ввода каждого элемента массива нажимается кнопка «Ввод»), 2-е текстовое поле - для вывода значения минимального элемента массива, 2-я кнопка «Вывод мин. элемента» - для вывода минимального элемента массива.
3. Создать страницу, содержащую приветствие в виде: «Сегодня - 10 сентября 2019 года, 13:01:25, вторник». Для этого используйте объекты Date и Array. Кроме того, страница, должна содержать форму, для ввода элементов массива и нахождения максимального элемента массива.
4. Создать страницу, содержащую приветствие в виде: «Сегодня - 10 сентября 2019 года, 13:01:25, вторник». Для этого используйте объекты Date и Array. Кроме того, страница, должна содержать форму, для ввода пользователем числа от 1 до 10. В сценарии создается массив, состоящий из 10 элементов, значения элементов лежат в диапазоне от 1 до 10, значения генерируются случайным образом с помощью функции Math.random(x). Если введенное пользователем число совпадает со значением элемента массива, то ему выводится соответствующее сообщение с указание порядкового номера элемента.
5. Создать страницу, содержащую приветствие в виде: «Сегодня - 10 сентября 2019 года, 13:01:25, вторник». Для этого используйте объекты Date и Array. Кроме того, страница, должна содержать форму, для нахождения гипотенузы треугольника по катету и прилежащему углу, либо противолежащему углу (реализовать оба случая на одной форме).
6. Создать страницу, содержащую приветствие в виде: «Сегодня - 10 сентября 2019 года, 13:01:25, вторник». Для этого используйте объекты Date и Array. Кроме того, страница, должна содержать форму, для ввода строки и нахождения в ней, введенной пользователем подстроки. Пользователю выводится сообщение о порядковом номере, с которого начинается подстрока в строке.
7. Создать страницу, содержащую приветствие в виде: «Сегодня - 10 сентября 2019 года, 13:01:25, вторник». Для этого используйте объекты Date и Array. Кроме того, страница, должна содержать форму, для ввода строки и удаления в ней пробелов. Пользователю выводится сообщение с получившейся строкой.

8. Создать страницу, содержащую приветствие в виде: «Сегодня - 10 сентября 2019 года, 13:01:25, вторник». Для этого используйте объекты Date и Array. Кроме того, страница, должна содержать форму, для ввода строки. Пользователю выводится строка – перевертыш исходной строки.

9. Создать страницу, содержащую приветствие в виде: «Сегодня - 10 сентября 2019 года, 13:01:25, вторник». Для этого используйте объекты Date и Array. Кроме того, страница, должна содержать форму, для ввода строки. Пользователю выводится сообщение о том, сколько раз повторяется в строке каждая из стоящих в ней букв.

10. Создать страницу, содержащую приветствие в виде: «Сегодня - 10 сентября 2019 года, 13:01:25, вторник». Для этого используйте объекты Date и Array. Кроме того, страница, должна содержать форму, для ввода числа. Пользователю выводится сообщение, является ли число простым или нет.

11. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 10 вещественных чисел. Найдите количество положительных, отрицательных и нулевых чисел. Используйте объект Array, Date и String.

12. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 10 вещественных чисел. Найдите среднее арифметическое положительных чисел. Используйте объект Array, Date и String.

13. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 10 вещественных чисел. Вычислите разность между максимальным и минимальным числами. Используйте объект Array, Date и String.

14. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 10 вещественных чисел. Вычислите сумму тех из них, которые кратны 5. Используйте объект Array, Date и String.

15. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 10 вещественных чисел. Определите: содержит ли последовательность хотя бы одну пару равных соседних элементов. Используйте объект Array, Date и String.

16. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 10 вещественных чисел. Определите: сколько раз меняется знак при переходе к следующему элементу. Используйте объект Array, Date и String.

17. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 10 вещественных чисел. Определите: является ли последовательность чисел возрастающей. Используйте объект Array, Date и String.

18. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 10 вещественных чисел. Вычислите разность между максимальными и минимальными числами. Используйте объект Array, Date и String.

19. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 9 вещественных чисел. Вычислите разность между 2 самыми минимальными числами. Используйте объект Array, Date и String.
20. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 10 вещественных чисел. Вычислите разность между 2 самыми максимальными числами. Используйте объект Array, Date и String.
21. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 11 вещественных чисел. Вычислите произведение модулей введенных чисел. Используйте объект Array, Date и String.
22. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 10 вещественных чисел. Вычислите сумму всех простых чисел. Используйте объект Array, Date и String.
23. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 10 вещественных чисел. Вычислите среднее геометрическое введенных чисел. Используйте объект Array, Date и String.
24. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 8 вещественных чисел. Определить является ли последовательность убывающей. Используйте объект Array, Date и String.
25. Написать программу на HTML и JavaScript для следующего задания: На экране в форме вводятся 10 вещественных чисел. Определить является ли последовательность арифметической прогрессией. Используйте объект Array, Date и String.

Задание №2

1. Создать пользовательский объект Поэт. При создании собственного объекта задайте не менее 3-х свойств и 2-х методов. Создать в скрипте 10 экземпляров данного объекта и вывести эти данные на страницу.
2. Создать пользовательский объект Абонент телефонной станции. При создании собственного объекта задайте не менее 4-х свойств и 3-х методов. Создать в скрипте 6 экземпляров данного объекта и вывести эти данные на страницу.
3. Создать пользовательский объект Животное. При создании собственного объекта задайте не менее 5-и свойств и 3-х методов. Создать в скрипте 4 экземпляров данного объекта и вывести эти данные на страницу.
4. Создать пользовательский объект Птица. При создании собственного объекта задайте не менее 3-х свойств и 3-х методов. Создать в скрипте 8 экземпляров данного объекта и вывести эти данные на страницу.
5. Создать пользовательский объект Художник. При создании собственного объекта задайте не менее 2-х свойств и 3-х методов. Создать в скрипте 7 экземпляров данного объекта и вывести эти данные на страницу.

6. Создать пользовательский объект Кухонный электроприбор. При создании собственного объекта задайте не менее 3-х свойств и 2-х методов. Создать в скрипте 10 экземпляров данного объекта и вывести эти данные на страницу.

7. Создать пользовательский объект Предмет мебели. При создании собственного объекта задайте не менее 4-х свойств и 3-х методов. Создать в скрипте 6 экземпляров данного объекта и вывести эти данные на страницу.

8. Создать пользовательский объект Доктор. При создании собственного объекта задайте не менее 5-и свойств и 3-х методов. Создать в скрипте 4 экземпляров данного объекта и вывести эти данные на страницу.

9. Создать пользовательский объект Насекомое. При создании собственного объекта задайте не менее 3-х свойств и 3-х методов. Создать в скрипте 8 экземпляров данного объекта и вывести эти данные на страницу.

10. Создать пользовательский объект Преподаватель. При создании собственного объекта задайте не менее 2-х свойств и 3-х методов. Создать в скрипте 7 экземпляров данного объекта и вывести эти данные на страницу.

11. Создать пользовательский объект Телеканал. При создании собственного объекта задайте не менее 3-х свойств и 2-х методов. Создать в скрипте 10 экземпляров данного объекта и вывести эти данные на страницу.

12. Создать пользовательский объект Клиент магазина. При создании собственного объекта задайте не менее 4-х свойств и 3-х методов. Создать в скрипте 6 экземпляров данного объекта и вывести эти данные на страницу.

13. Создать пользовательский объект Студент. При создании собственного объекта задайте не менее 5-и свойств и 3-х методов. Создать в скрипте 4 экземпляров данного объекта и вывести эти данные на страницу.

14. Создать пользовательский объект Менеджер. При создании собственного объекта задайте не менее 3-х свойств и 3-х методов. Создать в скрипте 8 экземпляров данного объекта и вывести эти данные на страницу.

15. Создать пользовательский объект Верхняя одежда. При создании собственного объекта задайте не менее 2-х свойств и 3-х методов. Создать в скрипте 7 экземпляров данного объекта и вывести эти данные на страницу.

16. Создать пользовательский объект Млекопитающее. При создании собственного объекта задайте не менее 3-х свойств и 2-х методов. Создать в скрипте 10 экземпляров данного объекта и вывести эти данные на страницу.

17. Создать пользовательский объект Фрукт. При создании собственного объекта задайте не менее 4-х свойств и 3-х методов. Создать в скрипте 6 экземпляров данного объекта и вывести эти данные на страницу.

18. Создать пользовательский объект Радиостанция. При создании собственного объекта задайте не менее 5-и свойств и 3-х методов. Создать в скрипте 4 экземпляров данного объекта и вывести эти данные на страницу.

19. Создать пользовательский объект Сотрудник компании. При создании собственного объекта задайте не менее 3-х свойств и 3-х методов. Создать в скрипте 8 экземпляров данного объекта и вывести эти данные на страницу.

20. Создать пользовательский объект Осветительный прибор. При создании собственного объекта задайте не менее 2-х свойств и 3-х методов. Создать в скрипте 7 экземпляров данного объекта и вывести эти данные на страницу.

21. Создать пользовательский объект Цветок. При создании собственного объекта задайте не менее 3-х свойств и 2-х методов. Создать в скрипте 10 экземпляров данного объекта и вывести эти данные на страницу.

22. Создать пользовательский объект Овощ. При создании собственного объекта задайте не менее 4-х свойств и 3-х методов. Создать в скрипте 6 экземпляров данного объекта и вывести эти данные на страницу.

23. Создать пользовательский объект Младенец. При создании собственного объекта задайте не менее 5-и свойств и 3-х методов. Создать в скрипте 4 экземпляров данного объекта и вывести эти данные на страницу.

24. Создать пользовательский объект Компьютерная программа. При создании собственного объекта задайте не менее 3-х свойств и 3-х методов. Создать в скрипте 8 экземпляров данного объекта и вывести эти данные на страницу.

25. Создать пользовательский объект Комплектующее устройство для компьютера. При создании собственного объекта задайте не менее 2-х свойств и 3-х методов. Создать в скрипте 7 экземпляров данного объекта и вывести эти данные на страницу.